



Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES

(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)

MoE, UGC & AICTE Approved; NAAC Accredited A++

Karunya Nagar, Coimbatore - 641 114, Tamil Nadu, India.

Division of Artificial Intelligence and Machine Learning

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

A MICRO PROJECT REPORT

SUBMITTED BY

SANTHOSA PRIYAN K A (URK25CS7100)

COURSE CODE

25CS209

COURSE NAME

PYTHON PROGRAMMING

APRIL 2026



Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES

(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)

MoE, UGC & AICTE Approved; NAAC Accredited A++

Karunya Nagar, Coimbatore - 641 114, Tamil Nadu, India.

Division of Artificial Intelligence and Machine Learning
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

Part – A

MICRO-PROJECT PROPOSAL

I. Details of the Students:

S.No	Name of the Student	Register Number
1	SINDHU PRIYA M	URK25CS7060
2	JAGAN S	URK25CS7006
3	SANTHOSAPRIYAN K A	URK25CS7100
4	MOHITH J A	URK25CS7135

II. Details of the Course related to the Micro-Project:

Course Title:	Python Programming
Course Code:	25CS209
Course Faculty:	Ms. Sophia
Academic Year:	2025-2026 [Even]
Semester:	II
Year & Batch:	5

III. Macro-Level and Meso-Level Sub-Tasks (COs) related to this Micro-Project:

Macro-Level Competency:	Employee Payroll Processing Application	
Course Outcomes:	CO 1	<i>Use</i> core Python concepts to build structured program components.
	CO 2	<i>Use</i> Python data structures to organize data.
	CO 3	<i>Develop</i> modular and reusable applications using Object Oriented Programming.
	CO 4	<i>Develop</i> File handling, GUI and database-connected applications
	CO 5	<i>Execute</i> image processing tasks using python libraries

Employee Payroll Processing Application

1.0 Rationale

In today's digital era, organizations require efficient systems to manage employee data and payroll processes accurately. Traditional payroll systems that rely on manual calculations are time-consuming, error-prone, and difficult to maintain. This creates a need for an automated payroll management system that ensures accuracy, efficiency, and reliability.

The Employee Payroll Processing Application is designed to simplify payroll operations by automating salary calculations, tax deductions, and record management. This project is significant as it reduces human errors in financial computations and provides a structured way to store employee data using a database system. By integrating a graphical user interface with backend database support, the system enables users to easily add, update, delete, and search employee records.

This project also plays an important role in enhancing the practical knowledge of students in Python programming. It involves the application of core programming concepts such as Object-Oriented Programming, database connectivity, and GUI development using Tkinter. Additionally, it helps students understand real-world application development and the importance of data management systems.

Overall, this project is beneficial for both learning purposes and practical implementation. It demonstrates how software solutions can improve efficiency in administrative tasks and can be further extended into a full-scale enterprise payroll system.

2.0 Literature Review

Payroll management systems have been widely studied and implemented in both academic and industrial environments. Various research articles, technical blogs, and software documentation highlight the importance of automating payroll processes to improve organizational efficiency. Traditional systems often involved manual record-keeping or basic spreadsheet tools, which lacked scalability and security.

Modern payroll systems use database management systems such as MySQL, Oracle, and SQLite to store and manage employee information. According to several technical

resources and Python documentation, SQLite is a lightweight and efficient database suitable for small to medium-scale applications. It is easy to integrate with Python, making it an ideal choice for student-level projects.

Graphical User Interface (GUI) development has also evolved significantly. Python's Tkinter library is one of the most commonly used tools for building simple desktop applications. It provides various widgets such as labels, buttons, and tables that allow developers to create interactive applications. Many educational resources emphasize Tkinter for beginners due to its simplicity and integration with Python.

Existing payroll applications in industries include advanced features such as tax slabs, employee benefits, attendance tracking, and report generation. However, for academic purposes, simplified payroll systems focus on core functionalities like employee data management and salary calculation.

This project is inspired by these existing systems and aims to implement a basic yet functional payroll system using Python, Tkinter, and SQLite. It bridges the gap between theoretical knowledge and practical implementation by demonstrating how software applications can be developed to solve real-world problems.

3.0 Proposed Methodology

The development of the Employee Payroll Processing Application follows a structured approach to ensure proper implementation and functionality. The methodology includes system design, development, testing, and deployment stages.

Initially, the system requirements are analyzed, including input fields such as employee ID, name, department, email, and salary. Based on these requirements, a database schema is designed using SQLite to store employee records efficiently.

Next, the graphical user interface is developed using Tkinter. The interface includes input forms, buttons for operations (Add, Update, Delete, Clear), and a table to display employee data. The design ensures user-friendly interaction and smooth navigation.

The backend logic is then implemented using Python. CRUD (Create, Read, Update, Delete) operations are developed to manage database records. When a user enters salary details, the system automatically calculates tax (10%) and net salary. These values are stored in the database along with other employee information.

The workflow of the system is as follows:

1. User enters employee details in the GUI
2. Data is validated and processed
3. Salary and tax are calculated
4. Data is stored in the SQLite database
5. Records are displayed in the table
6. User can update, delete, or search records

Finally, the application is tested to ensure all functionalities work correctly without errors. Any bugs or issues are fixed to improve performance and reliability.

This methodology ensures a systematic development process and results in a functional payroll application that meets the project requirements.

4.0 Tentative Action Plan *(Sequence of activities and time (Gantt Chart) required to complete the project.)*

S. No.	Details of the planned activities during the entire semester	Planned Start date	Planned Finish date	Name of the Responsible Team Member(s)
1	Requirement Analysis and Project Planning	01/02/2026	03/02/2026	JAGAN S
2	Study of Python GUI and SQLite Database	04/02/2026	08/02/2026	SINDHU PRIYA M
3	Designing System Flowchart and UI Layout	09/02/2026	12/02/2026	MOHITH J A
4	Database Design (Table creation, schema setup)	13/02/2026	15/02/2026	SANTHOSAPRIYAN K A
5	GUI Development (Forms, Buttons, Layout)	16/02/2026	22/02/2026	JAGAN S, MOHITH J A
6	Backend Coding	23/02/2026	25/02/2026	SINDHU PRIYA M, SANTHOSAPRIYAN K A
7	Integration of GUI with Database	01/03/2026	02/03/2026	JAGAN S, SINDHU PRIYA
8	Testing and Debugging	02/03/2026	03/03/2026	ALL Team members
9	Documentation and Report Preparation	04/03/2026	05/03/2026	ALL Team members
10	Final Review and Project Submission	07/03/2026	07/03/2026	ALL Team members

5.0 Tentative Resources (if any) Required *(major resources such as raw materials, some machining facilities, software etc.)*

S. No.	Name of Resource/Material	Broad Specifications	Qty.	Cost (INR)
1	Laptop /Computer	Minimum 8 GB RAM, Intel i5 or equivalent processor, Windows OS	1	Already Available
2	Python Software	Python 3.x, Tkinter library, SQLite3 database, IDE (VS Code / PyCharm)	1	Free(Open Source)

3	DB Browser for SQLite	Tool for viewing and managing SQLite database	1	Free
---	-----------------------	---	---	------

Employee Payroll Processing Application

A REAL TIME APPLICATION REPORT

Submitted by

SINDHU PRIYA M (URK25CS7060)

JAGAN S (URK25CS7006)

SANTHOSAPRIYAN K A (URK25CS7100)

MOHITH J A (URK25CS7135)



Division of Artificial Intelligence and Machine Learning

**KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed-to-be-under Sec-3 of the UGC Act,
1956) Karunya Nagar, Coimbatore - 641 114. INDIA**

APRIL 2026

OBJECTIVE OF THE PROJECT

The main objective of this project is to design and develop an efficient Employee Payroll Processing Application using Python. The system aims to automate the process of managing employee details, calculating salaries, and storing payroll data securely using a database.

This application helps reduce manual errors in salary calculation by automatically computing tax deductions and net salary. It provides a user-friendly graphical interface for easy data entry, updating, deletion, and retrieval of employee records.

The project also focuses on integrating GUI, database connectivity, and Python programming concepts to build a real-time application. It ensures proper management of employee payroll data in an organized and efficient manner.

FUNCTIONAL REQUIREMENTS

[MODULEWISE FEATURES OF THE PROJECT]

1. Employee Management Module

- Add new employee details (ID, Name, Department, Email, Salary)
- Update existing employee information
- Delete employee records
- Clear input fields

2. Payroll Calculation Module

- Automatic tax calculation (10% of salary)
- Net salary computation (Salary – Tax)

3. Search Module

- Search employee details using Employee ID
- Display specific record

4. Display Module

- View all employee records in tabular format
- Select and load data into form fields

5. Front End Description

The front end of the application is developed using Python's Tkinter library. It provides a modern graphical user interface with:

- Input forms for employee details
- Buttons for operations (Add, Update, Delete, Clear)
- Search bar for quick lookup
- Table view using Treeview widget

The interface is designed to be simple, user-friendly, and visually appealing.

6. Backend Description (Schema)

The backend is developed using SQLite database. The database stores employee payroll data.

Table Name: payroll

Column Name	Data Type	Description
id	TEXT (Primary Key)	Employee ID
name	TEXT	Employee Name
dept	TEXT	Department
email	TEXT	Email Address
salary	REAL	Gross Salary
tax	REAL	Tax Deduction (10%)
net	REAL	Net Salary

7.Database Interaction with Flow Diagram (Explanation)



8.Innovation of the Project

- Automatic salary and tax calculation
- Modern GUI design using Tkinter
- Real-time database interaction using SQLite
- Simple and efficient payroll management system
- Easy-to-use interface suitable for beginners

IMPLEMENTATION – CODING ONLY

```
import tkinter as tk

from tkinter import ttk, messagebox

import sqlite3


class ModernPayroll:

    def __init__(self, root):

        self.root = root

        self.root.title("Employee Payroll Processing Application")

        self.root.geometry("1100x650")

        self.root.configure(bg="#f4f7f6") # Light grey background


        # --- DATABASE CONNECTION ---

        self.conn = sqlite3.connect("payroll_system.db")

        self.cur = self.conn.cursor()

        self.cur.execute("""CREATE TABLE IF NOT EXISTS payroll

                               (id TEXT PRIMARY KEY, name TEXT, dept TEXT,

                               email TEXT, salary REAL, tax REAL, net REAL)""")

        self.conn.commit()


        # --- VARIABLES ---

        self.var_id = tk.StringVar()

        self.var_name = tk.StringVar()

        self.var_dept = tk.StringVar()

        self.var_email = tk.StringVar()

        self.var_salary = tk.StringVar()

        self.var_search = tk.StringVar()
```

```

# --- DESIGN: TOP HEADER ---

header = tk.Frame(self.root, bg="#2C3E50", height=70) # Dark blue/grey
header.pack(side="top", fill="x")

lbl_title = tk.Label(header, text="STAFF PAYROLL SYSTEM", font=("Trebuchet
MS", 22, "bold"),
                      fg="#ECF0F1", bg="#2C3E50")
lbl_title.pack(pady=15)

# --- DESIGN: LEFT SIDEBAR (INPUT FORM) ---

form_frame = tk.Frame(self.root, bg="white", bd=0, relief="flat")
form_frame.place(x=20, y=90, width=380, height=540)

# Shadow effect (simulated)
tk.Label(form_frame, text="Employee Registration", font=("Arial", 14, "bold"),
        bg="white", fg="#2C3E50").pack(pady=20)

inner_frame = tk.Frame(form_frame, bg="white")
inner_frame.pack(padx=20)

# Helper function for labels and entries
def create_entry(label_text, variable):
    lbl = tk.Label(inner_frame, text=label_text, font=("Arial", 10), bg="white",
fg="#7F8C8D")
    lbl.pack(anchor="w", pady=(10, 0))
    entry = tk.Entry(inner_frame, textvariable=variable, font=("Arial", 12), bd=0,
highlightthickness=1, highlightbackground="#BDC3C7",
bg="#FDFEFE")
    entry.pack(fill="x", ipady=5)

create_entry("Employee ID", self.var_id)

```

```

create_entry("Full Name", self.var_name)
create_entry("Department", self.var_dept)
create_entry("Email Address", self.var_email)
create_entry("Base Salary ($)", self.var_salary)

# --- BUTTONS ---

btn_frame = tk.Frame(form_frame, bg="white")
btn_frame.pack(pady=30)

# Styling buttons with specific colors
def create_btn(text, color, cmd, col):
    btn = tk.Button(btn_frame, text=text, command=cmd, bg=color, fg="white",
                    font=("Arial", 10, "bold"), width=8, bd=0, cursor="hand2")
    btn.grid(row=0, column=col, padx=5)

create_btn("ADD", "#27AE60", self.add_record, 0) # Green
create_btn("UPDATE", "#2980B9", self.update_record, 1) # Blue
create_btn("DELETE", "#E74C3C", self.delete_record, 2) # Red
create_btn("CLEAR", "#95A5A6", self.clear_form, 3) # Grey

# --- DESIGN: RIGHT SIDE (SEARCH & TABLE) ---

table_frame = tk.Frame(self.root, bg="#f4f7f6")
table_frame.place(x=420, y=90, width=650, height=540)

# Search Bar

search_frame = tk.Frame(table_frame, bg="#f4f7f6")
search_frame.pack(fill="x", pady=(0, 10))

tk.Entry(search_frame, textvariable=self.var_search, font=("Arial", 12), width=30,
        bd=0, highlightthickness=1).pack(side="left", padx=5, ipady=3)

```

```
tk.Button(search_frame, text="Search ID", command=self.search_record,  
bg="#34495E",
```

```
fg="white", bd=0, padx=10).pack(side="left", padx=5)
```

```
tk.Button(search_frame, text="Show All", command=self.fetch_data,  
bg="#95A5A6",
```

```
fg="white", bd=0, padx=10).pack(side="left")
```

```
# Table Styling (Treeview)
```

```
style = ttk.Style()
```

```
style.configure("Treeview", font=("Arial", 10), rowheight=30,  
background="#FFFFFF")
```

```
style.configure("Treeview.Hheading", font=("Arial", 11, "bold"))
```

```
style.map("Treeview", background=[('selected', '#3498DB')]) # Blue selection
```

```
self.employee_table = ttk.Treeview(table_frame, columns=("id", "name", "dept",  
"salary", "net"))
```

```
self.employee_table.heading("id", text="ID")
```

```
self.employee_table.heading("name", text="Name")
```

```
self.employee_table.heading("dept", text="Dept")
```

```
self.employee_table.heading("salary", text="Gross")
```

```
self.employee_table.heading("net", text="Net Salary")
```

```
self.employee_table['show'] = 'headings'
```

```
# Column widths
```

```
self.employee_table.column("id", width=50)
```

```
self.employee_table.column("salary", width=80)
```

```
self.employee_table.column("net", width=100)
```

```
self.employee_table.pack(fill="both", expand=True)
```

```
self.employee_table.bind("<ButtonRelease-1>", self.get_cursor)
```

```
self.fetch_data()
```

```
# --- LOGIC & DATABASE FUNCTIONS ---
```

```
def add_record(self):
```

```
    if self.var_id.get() == "" or self.var_salary.get() == "":
```

```
        messagebox.showerror("Error", "Required fields: ID and Salary")
```

```
    return
```

```
    try:
```

```
        sal = float(self.var_salary.get())
```

```
        tax = sal * 0.10 # 10% Tax calculation
```

```
        net = sal - tax
```

```
        self.cur.execute("INSERT INTO payroll VALUES (?, ?, ?, ?, ?, ?)",
```

```
                           (self.var_id.get(), self.var_name.get(), self.var_dept.get(),
```

```
                           self.var_email.get(), sal, tax, net))
```

```
        self.conn.commit()
```

```
        self.fetch_data()
```

```
        messagebox.showinfo("Success", "Employee Added")
```

```
        self.clear_form()
```

```
    except Exception as e:
```

```
        messagebox.showerror("Error", f"Could not add: {str(e)}")
```

```
def fetch_data(self):
```

```
    self.cur.execute("SELECT id, name, dept, salary, net FROM payroll")
```

```
    rows = self.cur.fetchall()
```

```
    self.employee_table.delete(*self.employee_table.get_children())
```

```
    for row in rows:
```

```
        self.employee_table.insert("", tk.END, values=row)
```

```

def get_cursor(self, ev):
    cursor_row = self.employee_table.focus()
    contents = self.employee_table.item(cursor_row)
    row = contents['values']
    if row:
        self.var_id.set(row[0])
        self.var_name.set(row[1])
        self.var_dept.set(row[2])
        # Fetch full details from DB
        self.cur.execute("SELECT email, salary FROM payroll WHERE id=?", (row[0],))
        res = self.cur.fetchone()
        self.var_email.set(res[0])
        self.var_salary.set(res[1])

def update_record(self):
    sal = float(self.var_salary.get())
    tax = sal * 0.10
    net = sal - tax
    self.cur.execute("UPDATE payroll SET name=?, dept=?, email=?, salary=?, tax=?,
net=? WHERE id=?",
                    (self.var_name.get(), self.var_dept.get(), self.var_email.get(),
                     sal, tax, net, self.var_id.get()))
    self.conn.commit()
    self.fetch_data()
    messagebox.showinfo("Success", "Record Updated")

def delete_record(self):
    self.cur.execute("DELETE FROM payroll WHERE id=?", (self.var_id.get(),))
    self.conn.commit()
    self.fetch_data()

```

```
self.clear_form()
```

```
def search_record(self):
```

```
    self.cur.execute("SELECT id, name, dept, salary, net FROM payroll WHERE id=?",  
(self.var_search.get(),))
```

```
    row = self.cur.fetchone()
```

```
    self.employee_table.delete(*self.employee_table.get_children())
```

```
    if row:
```

```
        self.employee_table.insert("", tk.END, values=row)
```

```
    else:
```

```
        messagebox.showwarning("Warning", "ID not found")
```

```
def clear_form(self):
```

```
    self.var_id.set("")
```

```
    self.var_name.set("")
```

```
    self.var_dept.set("")
```

```
    self.var_email.set("")
```

```
    self.var_salary.set("")
```

```
    self.var_search.set("")
```

```
if __name__ == "__main__":
```

```
    root = tk.Tk()
```

```
    obj = ModernPayroll(root)
```

```
    root.mainloop()
```


OUTPUT SCREENSHOT

The screenshot displays the 'Enterprise Payroll Management System' window. The main title is 'STAFF PAYROLL SYSTEM'. On the left, there is an 'Employee Registration' form with fields for Employee ID, Full Name, Department, Email Address, and Base Salary (\$). Below the form are buttons for ADD, UPDATE, DELETE, and CLEAR. On the right, there is a table with columns for ID, Name, Dept, Gross, and Net Salary. The table contains one row of data for Employee ID 1234, Name 'sindhu', Dept 'aiml', Gross '20000.0', and Net Salary '18000.0'. Above the table are search and display controls: a search bar, a 'Search ID' button, and a 'Show All' button.

ID	Name	Dept	Gross	Net Salary
1234	sindhu	aiml	20000.0	18000.0

FUTURE ENHACEMENTS

1. Add login authentication system
2. Generate salary slips (PDF format)
3. Add multiple tax slabs
4. Cloud database integration
5. Export data to Excel
6. Add employee attendance and bonus calculation
7. Improve UI using advanced frameworks



Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES

(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)

MoE, UGC & AICTE Approved; NAAC Accredited A++

Karunya Nagar, Coimbatore - 641 114, Tamil Nadu, India.

School of Computer Science and Technology

Division of Computer Science and Engineering

Part - B

Evaluation Criteria for Micro-Project

(to be monitored fortnightly)

I. Details of the Students:

S.No	Name of the Student	Register Number
1	SINDHU PRIYA M	URK25CS7060
2	JAGAN S	URK25CS7006
3	SANTHOSAPRIYAN K A	URK25CS7100
4	MOHITH J A	URK25CS7135

II. Details of the Course related to the Micro-Project:

Course Title:	Python Programming
Course Code:	25CS209
Course Faculty:	Ms. Sophia
Designation:	Professor/Associate Professor/Assistant Professor (SG)/Assistant Professor
Course Coordinator:	Dr. S. Stewart Kirubakaran
Academic Year:	2025-2026 [Even]
Semester:	II
Year & Batch:	I & 5

Title of the Micro-Project	Employee Payroll Processing Application
-----------------------------------	---

III. Rubric for Assessment of Micro-Project:

S. No.	Criteria	Sub - Criteria	Very Good (5 marks)	Good (3 marks)	Average (2 marks)	Needs Improvement (1 mark)	Max. Marks
1	Attainment of CO (5 marks)	Relevance to the COs	Addresses 3 or more COs	Addresses only 2 COs	Addresses at-least 1CO	Not addressing any COs	
2	Working Model / Software	GUI	User friendly GUI, Handling all kinds of input and output	Missing few requirements	Improper GUI	Incomplete GUI	
3		Integration of front end and back end	Proper integration of GUI and database (CRUD Operations)	Successful storage and retrieval of data	Creation of relational database yet no functionalities	Database connection accomplished, but no table created	
4		Innovation	Innovation in application Implemented	Innovation in GUI design only	No innovation	Copied from internet	
5	Individual Technical Assessment (15 marks)	Implementation Quality & Technical Correctness	Code is correct, modular, efficient, and handles edge cases. No runtime errors.	Works for basic cases but fails edge cases or has inefficient logic.	Frequent errors. Poor structure and inconsistent logic.	Code does not run or produces incorrect output.	
6		Technical Explanation & Justification	Clearly explains logic, design choices, tools, and trade-offs with technical accuracy.	Basic explanation; struggles with deeper "why" questions.	Superficial explanation; relies on memorized statements.	Cannot explain their own work.	

7		Viva Questions	Answers are consistently correct, precise, and technically sound	Answers are partially correct but lack depth	Answers are incomplete, inconsistent, or vague	Mostly incorrect or no answers	
8	Project Report (5 marks)	Report	Well formatted, content coverage, grammar and spelling	Content coverage, grammar and spelling	Content coverage	Lack of content	
Total Marks							4

COs Attained:

CO 1:	<u>Use</u> core Python concepts to build structured program components.
CO 2:	<u>Use</u> Python data structures to organize and preprocess data.
CO 3:	<u>Develop</u> modular and reusable applications using OOP.
CO 4:	<u>Develop</u> file handling, GUI and database-connected applications.
CO 5:	<u>Execute</u> image processing tasks using python libraries.

Competency Attained: Yes / No / Partially

IV. Assessment of Micro-Project:

Process Assessment	Product Assessment			Total Marks
Part A	Part B			
Attainment of COs (5 marks)	Working Model/Software (15 marks)	Individual Technical Assessment (15 marks)	Project Report (5 marks)	40

Comments/Suggestions about teamwork/leadership/inter-personal communication (if any)

.....

Any Other Comment

.....
...

Signature of the Faculty-in-charge

,